

EveryMovie — API Documentation

Base URL: <https://everymovie.onrender.com>

Authentication: Session-based (Django). Protected endpoints require the user to be logged in via [/login/](#). Unauthenticated requests to protected endpoints return **401 Unauthorized**.

Response Format: All API endpoints return **application/json**.

Table of Contents

1. [GET /genres/](#)
 2. [GET /discover/](#)
 3. [GET /search/](#)
 4. [GET /api/trending/](#)
 5. [GET /api/lists/](#)
 6. [POST /lists/create/](#)
 7. [POST /lists/<id>/rename/](#)
 8. [POST /lists/<id>/delete/](#)
 9. [POST /lists/<id>/add/](#)
 10. [POST /lists/item/<id>/remove/](#)
 11. [GET /movie/<id>/](#)
 12. [Authentication Endpoints](#)
 13. [Error Codes](#)
 14. [Data Models](#)
-

1. GET /genres/

Returns all TMDb movie genres plus dynamically detected genre pairs (two-genre combinations observed in popular and top-rated films).

Parameters

None.

Example Request

```
GET /genres/
```

Example Response (200 OK)

```
{
  "genres": [
```

```

    { "id": 28, "name": "Action" },
    { "id": 12, "name": "Adventure" },
    { "id": 16, "name": "Animation" },
    { "id": 35, "name": "Comedy" },
    { "id": 80, "name": "Crime" },
    { "id": 99, "name": "Documentary" },
    { "id": 18, "name": "Drama" },
    { "id": 10751, "name": "Family" },
    { "id": 14, "name": "Fantasy" },
    { "id": 36, "name": "History" },
    { "id": 27, "name": "Horror" },
    { "id": 10402, "name": "Music" },
    { "id": 9648, "name": "Mystery" },
    { "id": 10749, "name": "Romance" },
    { "id": 878, "name": "Science Fiction" },
    { "id": 53, "name": "Thriller" },
    { "id": 10752, "name": "War" },
    { "id": 37, "name": "Western" }
  ],
  "pairs": [
    {
      "id": "28,35",
      "ids": [28, 35],
      "label": "Action/Comedy"
    },
    {
      "id": "28,80",
      "ids": [28, 80],
      "label": "Action/Crime"
    }
  ]
}
```

Notes

- `pairs` are derived by sampling popular and top-rated movies from TMDb and extracting all two-genre combinations that actually appear.
- The `id` field on pairs is a comma-separated string of genre IDs, directly usable as the `genre` parameter in `/discover/`.

2. GET /discover/

Discover movies filtered by genre and country, with configurable sorting. Uses a dynamic quality filter for rating-based sorts to ensure meaningful results.

Parameters

Parameter	Type	Required	Default	Description
-----------	------	----------	---------	-------------

Parameter	Type	Required	Default	Description
genre	string	Yes	—	TMDB genre ID(s). For pairs, use comma-separated IDs (e.g. 28,35).
country	string	Yes	—	ISO 3166-1 alpha-2 country code (e.g. US, GB, FR).
sort	string	No	rating	Sort order: rating (by vote average) or az (alphabetical).
page	string	No	1	Page number for pagination (TMDB pages).

Example Request

```
GET /discover/?genre=28&country=US&sort=rating&page=1
```

Example Response (200 OK)

```
{
  "page": 1,
  "results": [
    {
      "id": 278,
      "title": "The Shawshank Redemption",
      "overview": "Imprisoned in the 1940s for the double murder...",
      "poster_path": "/9cjIGRiQlMpHANfNbAN6sM1wRF0.jpg",
      "vote_average": 8.7,
      "vote_count": 26500,
      "release_date": "1994-09-23",
      "genre_ids": [18, 80]
    }
  ],
  "total_pages": 5,
  "total_results": 95,
  "_quality": {
    "vote_count_gte_used": 2000,
    "total_results_estimate": 15000
  }
}
```

Error Responses

Status	Condition	Body
400	Missing genre or country	{"error": "Missing genre or country"}

Notes

- The _quality metadata object is included for rating sorts, showing the minimum vote count threshold that was applied to filter out low-quality entries.

- Poster images can be loaded from https://image.tmdb.org/t/p/w500{poster_path}.

3. GET /search/

Free-text movie search powered by TMDB's search API.

Parameters

Parameter	Type	Required	Default	Description
q	string	Yes	—	Search query string (movie title).
page	string	No	1	Page number for pagination.

Example Request

```
GET /search/?q=inception&page=1
```

Example Response (200 OK)

```
{
  "page": 1,
  "results": [
    {
      "id": 27205,
      "title": "Inception",
      "overview": "Cobb, a skilled thief who commits corporate espionage...",
      "poster_path": "/ljsZTbVsrQSqZgWeep2B1QiDKuh.jpg",
      "vote_average": 8.4,
      "release_date": "2010-07-15",
      "genre_ids": [28, 878, 12]
    }
  ],
  "total_pages": 1,
  "total_results": 4
}
```

Error Responses

Status	Condition	Body
400	Missing or empty q parameter	<code>{"results": [], "error": "Missing query parameter 'q'."}</code>
502	TMDB service unreachable	<code>{"results": [], "error": "Search service unavailable."}</code>

4. GET /api/trending/

Returns currently popular movies, optionally filtered by country.

Parameters

Parameter	Type	Required	Default	Description
country	string	No	—	ISO 3166-1 alpha-2 country code to filter by origin country.
page	string	No	1	Page number for pagination.

Example Request

```
GET /api/trending/?country=GB&page=1
```

Example Response (200 OK)

```
{
  "page": 1,
  "results": [
    {
      "id": 634649,
      "title": "Spider-Man: No Way Home",
      "overview": "Peter Parker is unmasked and no longer able to...",
      "poster_path": "/1g0dhYtq4irTY1GPXvft6k4YLjm.jpg",
      "vote_average": 8.0,
      "release_date": "2021-12-15",
      "genre_ids": [28, 12, 878]
    }
  ],
  "total_pages": 500,
  "total_results": 10000
}
```

Error Responses

Status	Condition	Body
502	TMDB service unreachable	{"results": [], "error": "Trending service unavailable."}

5. GET /api/lists/

Returns all movie lists belonging to the authenticated user.

Authentication

Required. Returns 401 if not logged in.

Parameters

None.

Example Request

```
GET /api/lists/  
Cookie: sessionid=abc123...
```

Example Response (200 OK)

```
{  
  "lists": [  
    {  
      "id": 1,  
      "name": "Watch Later",  
      "item_count": 5,  
      "created_at": "2026-03-01T14:30:00+00:00"  
    },  
    {  
      "id": 2,  
      "name": "Favourites",  
      "item_count": 12,  
      "created_at": "2026-02-28T09:15:00+00:00"  
    }  
  ]  
}
```

Error Responses

Status	Condition	Body
401	Not authenticated	{"detail": "Authentication required."}

6. POST /lists/create/

Create a new movie list for the authenticated user.

Authentication

Required. Redirects to login page if not authenticated.

Parameters (form body)

Parameter	Type	Required	Description
-----------	------	----------	-------------

Parameter	Type	Required	Description
<code>name</code>	string	Yes	Name for the new list. Must be unique per user.

Example Request

```
POST /lists/create/  
Content-Type: application/x-www-form-urlencoded  
  
name=Watch+Later
```

Response

Redirects to the referring page (or `/my-lists/`). Success/error messages are set via Django's messages framework.

Error Conditions

- Empty name → error message "List name cannot be empty."
- Duplicate name → error message "You already have a list with that name."

7. POST `/lists/<id>/rename/`

Rename an existing list.

Authentication

Required.

Parameters (form body)

Parameter	Type	Required	Description
<code>name</code>	string	Yes	New name for the list.

Example Request

```
POST /lists/1/rename/  
Content-Type: application/x-www-form-urlencoded  
  
name=Top+Picks
```

Response

Redirects to `/my-lists/`. Sets success or error message.

Error Conditions

- Empty name → "Name cannot be empty."
- Duplicate name → "You already have a list with that name."
- List not found or not owned by user → 404.

8. POST /lists/<id>/delete/

Delete a movie list and all its items.

Authentication

Required.

Example Request

```
POST /lists/1/delete/
```

Response

Redirects to </my-lists/>. Sets success message "List deleted."

Error Conditions

- List not found or not owned by user → 404.

9. POST /lists/<id>/add/

Add a movie to a list.

Authentication

Required.

Parameters (form body)

Parameter	Type	Required	Description
tmdb_id	int	Yes	TMDB movie ID.
title	string	No	Movie title (defaults to "Untitled").
poster_path	string	No	TMDB poster path (e.g. /abc123.jpg).
next	string	No	URL to redirect to after adding (defaults to /).

Example Request

```
POST /lists/1/add/
Content-Type: application/x-www-form-urlencoded
```

```
tmbd_id=27205&title=Inception&poster_path=/ljsZTbVsrQsqZgWeep2B1QiDKuh.jpg&next=/movie/27205/
```

Response

Redirects to **next** URL. Sets success or error message.

Error Conditions

- Missing **tmbd_id** → "Missing movie id."
- Duplicate movie in same list → "'Inception' is already in 'Watch Later'."

10. POST /lists/item/<id>/remove/

Remove a movie from a list.

Authentication

Required.

Example Request

```
POST /lists/item/42/remove/
```

Response

Redirects to **/my-lists/** (or **next** parameter). Sets success message "Movie removed."

Error Conditions

- Item not found or not owned by user → 404.

11. GET /movie/<id>/

Renders the movie detail page (HTML). Fetches full movie details and credits from TMDB.

Parameters

Parameter	Type	Required	Description
id (URL path)	int	Yes	TMDB movie ID.

Example Request

```
GET /movie/27205/
```

Response

Returns an HTML page containing:

- Movie title, overview, poster, backdrop
- Director name, top 10 cast members
- Genres, runtime, release date, vote average
- "Add to list" dropdown (for authenticated users)

Error Conditions

- Movie not found on TMDB → renders page with error message "Movie not found."

12. Authentication Endpoints

POST /signup/

Register a new user account. Passwords are hashed with PBKDF2-SHA256 with a random salt.

Parameters (form body):

Parameter	Type	Required	Description
username	string	Yes	Unique username.
email	string	Yes	Email address.
password	string	Yes	Password.
password2	string	Yes	Password confirmation (must match password).

Error Conditions:

- Missing fields → "All fields are required."
- Passwords don't match → "Passwords do not match."
- Username taken → "Username already taken."
- Email in use → "Email already in use."

Success: Creates user, logs them in, redirects to /.

POST /login/

Authenticate an existing user.

Parameters (form body):

Parameter	Type	Required	Description
username	string	Yes	Username.

Parameter	Type	Required	Description
password	string	Yes	Password.

Error Conditions:

- Invalid credentials → "Invalid username or password."

Success: Logs user in, redirects to /.

GET /logout/

Logs the current user out and redirects to /.

13. Error Codes

HTTP Status	Meaning	Used By
200	Success	All endpoints
400	Bad Request — missing or invalid parameters	/discover/, /search/
401	Unauthorized — user not logged in	/api/lists/
404	Not Found — resource doesn't exist or not owned by user	List CRUD endpoints, /movie/<id>/
502	Bad Gateway — upstream TMDb service unreachable	/search/, /api/trending/

All error responses return JSON with an "error" or "detail" key describing the problem.

14. Data Models

MovieList

Field	Type	Description
id	Integer (PK)	Auto-generated primary key.
user	ForeignKey	The owning user (Django auth User model).
name	CharField	List name (max 120 chars). Unique per user.
created_at	DateTime	Auto-set on creation.

MovieListItem

Field	Type	Description
id	Integer (PK)	Auto-generated primary key.

Field	Type	Description
movie_list	ForeignKey	Parent MovieList. Cascade delete.
tmdb_id	Integer	TMDB movie ID. Unique per list.
title	CharField	Movie title (max 300 chars).
poster_path	CharField	TMDB poster image path.
added_at	DateTime	Auto-set on creation.

External API

This application consumes the [TMDB API v3](#). The following TMDB endpoints are used:

TMDB Endpoint	Used By
GET /genre/movie/list	/genres/
GET /discover/movie	/genres/, /discover/, /api/trending/
GET /search/movie	/search/
GET /movie/{id}	/movie/<id>/
GET /movie/{id}/credits	/movie/<id>/

TMDB data is used under their [API Terms of Use](#).